



SIMATS ENGINEERING
SAVEETHA INSTITUTE OF MEDICAL AND
TECHNICAL SCIENCES



CSA0613-Design & Analysis of Algorithm for complexity Analysis

Analysis of the Logarithmic Recurrence $T(n)=T(n/2)+\log n$ using the Substitution Method

Guided by:

Dr.F.Mary Harin Fernandez

Presented by:

A.Bhavya [192424417]



Engineer to Excel

INTRODUCTION



The **Substitution Method** is one of the most commonly used techniques for solving recurrence relations. It works by repeatedly expanding the recurrence until a recognizable pattern appears, then simplifying the resulting expression to determine the asymptotic time complexity.

The recurrence

$$T(n) = T(n/2) + \log n$$

describes algorithms that divide the input into two equal halves at each recursive step while performing logarithmic work at every level. Such recurrences are common in divide-and-conquer algorithms, search indexing, and recursive optimization techniques.

Problem Statement



Analyze the recurrence relation

$$T(n)=T(n/2)+\log n$$

using the **Substitution Method**.

Objectives:

- Expand the recurrence step-by-step.
- Identify the mathematical pattern.
- Simplify the recurrence.
- Determine the asymptotic time complexity.
- Interpret the efficiency of the algorithm.

Algorithm (Substitution Method)

Algorithm SolveRecurrence(n)

Input : Size n

Output: Time Complexity

Step 1: Write recurrence

$$T(n) = T(n/2) + \log n$$

Step 2: Expand recursively

Step 3: Continue expansion until base case

Step 4: Count total recursive levels

Step 5: Add logarithmic work done at each level

Step 6: Simplify summation

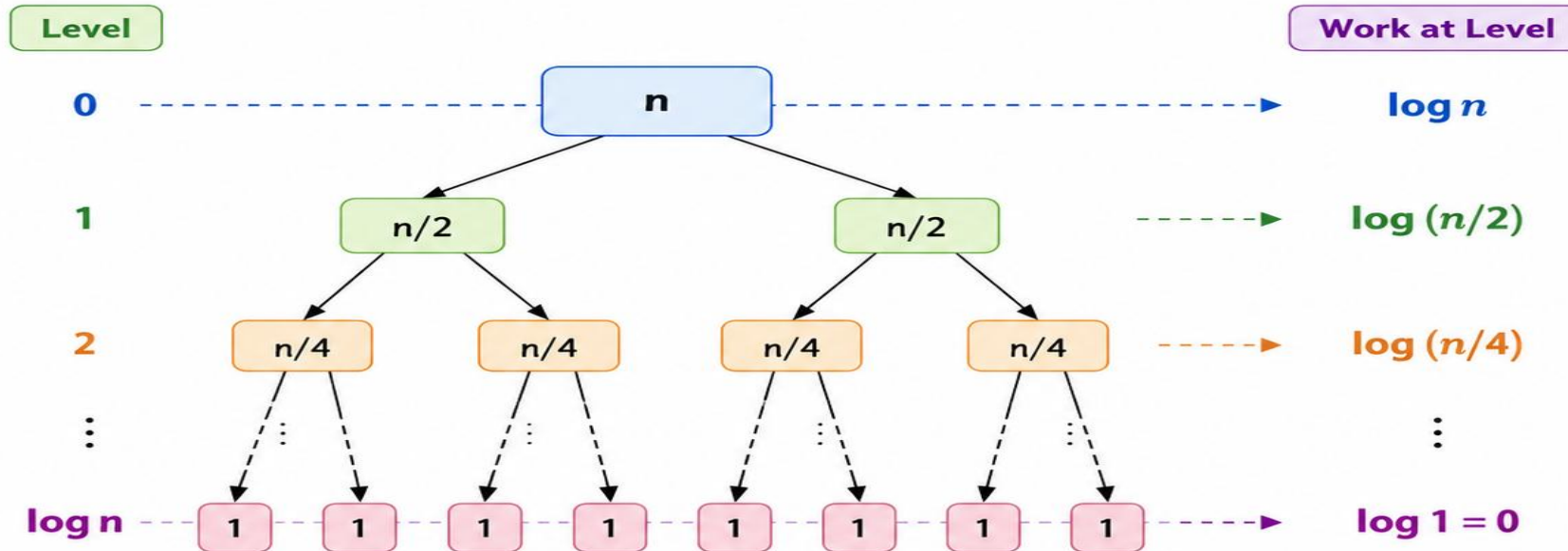
Step 7: Derive Big-O complexity

End

Architecture Diagram

Recursion Tree for $T(n) = T(n/2) + \log n$

At each level, the problem size halves and work done is logarithmic.



Summary:

- Number of levels = $\log n$
- Work at level $i = \log(n/2^i)$
- Total work = $\sum_{i=0}^{\log n - 1} \log \frac{n}{2^i}$

$$\begin{aligned}
 &= \sum_{i=0}^{\log n - 1} (\log n - i) \\
 &= (\log n) + (\log n - 1) + (\log n - 2) + \dots + 1 \\
 &= \frac{(\log n)(\log n + 1)}{2} \\
 &= O((\log n)^2)
 \end{aligned}$$

Hence,

$$T(n) = O((\log n)^2)$$

Legend:

- Root Problem
- Level 1 Subproblems
- Level 2 Subproblems
- Base Case ($n = 1$)

Implementation Code

```
main.py  [Full Screen] [Theme] [Share] [Run]
1  import math
2
3  def recurrence(n):
4      if n == 1:
5          return 1
6      return recurrence(n // 2) + math.log2(n)
7
8  n = 16
9  result = recurrence(n)
10
11 print("Value of T(", n, ") =", result)
```

Output

Output

```
Value of T( 16 ) = 11.0
```

```
=== Code Execution Successful ===
```

Complexity Analysis

Parameter	Complexity
Recurrence	$(T(n)=T(n/2)+\log n)$
Recursive Levels	$(\log n)$
Work per Level	$(\log n, \log(n/2), \log(n/4))$
Total Work	$(\log n)^2$
Time Complexity	$(O(\log n)^2)$
Space Complexity (recursion stack)	$(O(\log n))$

Conclusion

The recurrence $T(n)=T(n/2)+\log n$ was solved using the substitution method by expanding the recurrence, identifying the summation pattern, and simplifying it mathematically. The final time complexity is **$O(\log n)^2$** , indicating that the algorithm remains highly efficient because the input size halves at every recursive step while only logarithmic work is performed at each level.

References

- Thomas H. Cormen, Charles E. Leiserson, Ronald L. Rivest, and Clifford Stein, **Introduction to Algorithms**, 4th Edition, MIT Press.
- Ellis Horowitz, Sartaj Sahni, and Sanguthevar Rajasekaran, **Fundamentals of Computer Algorithms**, Universities Press.
- Alfred V. Aho, John E. Hopcroft, and Jeffrey D. Ullman, **The Design and Analysis of Computer Algorithms**, Addison-Wesley.
- Donald E. Knuth, **The Art of Computer Programming, Volume 1: Fundamental Algorithms**, Addison-Wesley.
- Michael T. Goodrich, Roberto Tamassia, and Michael H. Goldwasser, **Data Structures and Algorithms in Python**, Wiley

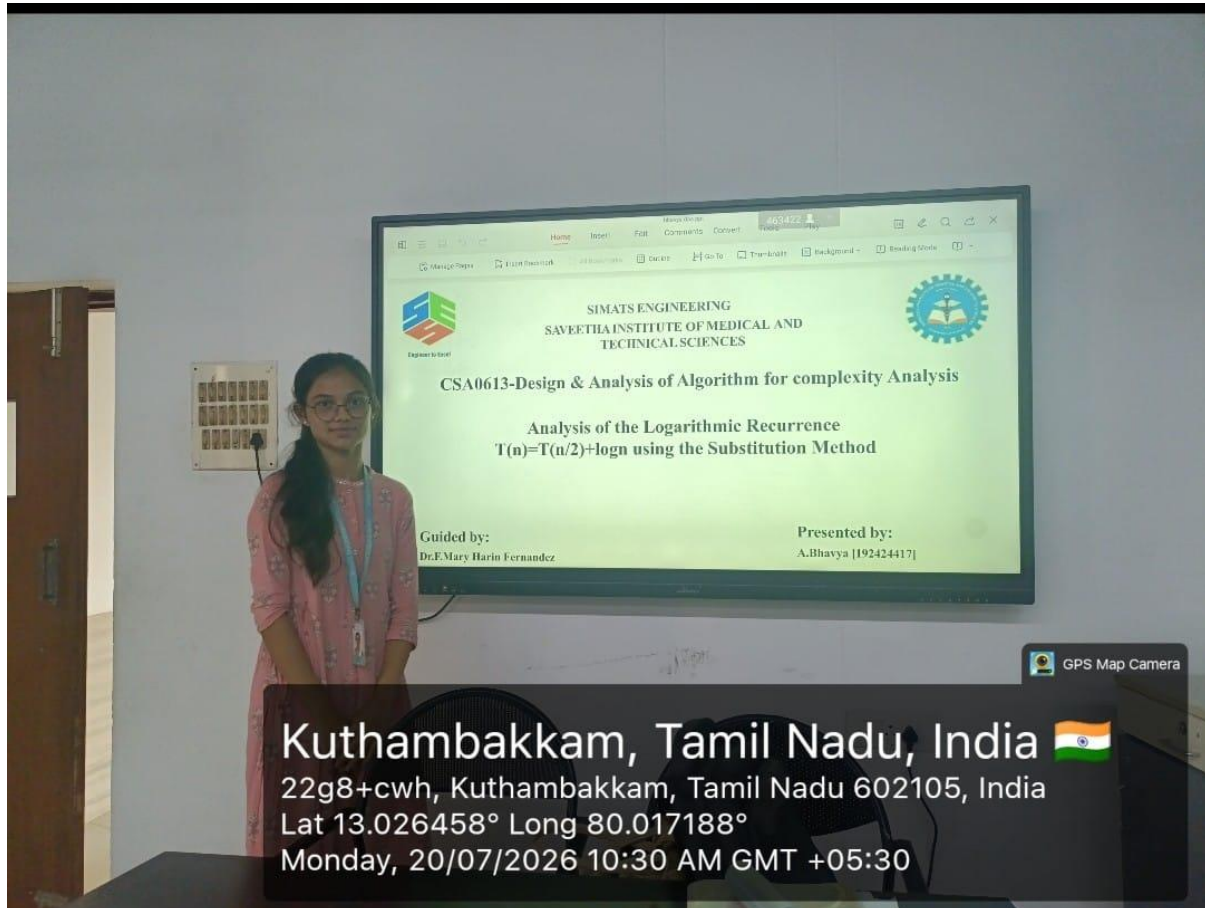
THANK YOU

CO1-AT3

Technical presentation

Name: A.Bhavya

Reg no:192424417



The Substitution Method is a technique used to solve recurrence relations by repeatedly expanding the recursive term until the base case is reached. In this topic, the recurrence represents an algorithm that divides the input into two equal halves at each recursive call while performing logarithmic work at every level. By expanding the recurrence step by step, a pattern is identified, and the recursion continues until the input size becomes 1. The total work is obtained by adding the logarithmic work performed at each recursive level. After simplifying the summation, the final time complexity is , which indicates that the algorithm is highly efficient because the input size decreases rapidly while only a small amount of work is performed at each step.